

Caio Xavier da Silva

**Análise Estática Independente de Linguagem:
Um Framework Genérico Orientado à Geração de
Padrões Sintáticos**

São Paulo

2026

Caio Xavier da Silva

Análise Estática Independente de Linguagem: Um Framework Genérico Orientado à Geração de Padrões Sintáticos

Proposta de Trabalho de Conclusão de Curso apresentado ao SENAC Santo Amaro como requisito parcial para aprovação na disciplina de TCC1 do Bacharelado em Ciência da Computação.

SENAC – Serviço Nacional de Aprendizagem Comercial
Unidade Santo Amaro
Bacharelado em Ciência da Computação

Orientador: Celso Vital Crivelaro

São Paulo
2026

Resumo

Este documento apresenta a proposta inicial do Trabalho de Conclusão de Curso (TCC1) desenvolvido no SENAC Santo Amaro. O objetivo deste trabalho é [descrever brevemente o objetivo principal do TCC]. A justificativa para este estudo baseia-se em [descrever brevemente a justificativa]. A revisão bibliográfica abrange [mencionar os principais tópicos]. Como solução proposta, pretende-se [descrever brevemente a solução]. Este trabalho está organizado em três capítulos que apresentam a introdução ao tema, a revisão bibliográfica que fundamenta a pesquisa, o desenvolvimento com a solução proposta e o cronograma de trabalho, seguidos das referências bibliográficas.

Palavras-chave: palavra1. palavra2. palavra3. palavra4. palavra5.

Sumário

1	INTRODUÇÃO	4
1.1	Contexto	6
1.2	Justificativa	7
1.3	Hipótese	8
1.4	Objetivo	9
1.4.1	Objetivo principal	9
1.4.2	Objetivos secundários	9
2	REVISÃO BIBLIOGRÁFICA	10
2.1	Inteligência Artificial	10
2.1.1	Linguagem Natural	10
2.1.2	Modelos de Linguagens de Grande Porte (LLM)	11
2.1.3	Aprendizado de regras baseado em LLMs	12
2.2	Metodologia da pesquisa bibliográfica	13
2.3	Fundamentos	14
2.4	Trabalhos relacionados	14
2.4.1	Trabalho 1: [Título do trabalho]	15
2.4.2	Trabalho 2: [Título do trabalho]	15
2.4.3	Trabalho 3: [Título do trabalho]	15
3	DESENVOLVIMENTO	16
3.1	Solução proposta	16
3.2	Cronograma de trabalho	16
	REFERÊNCIAS BIBLIOGRÁFICAS	18

1 Introdução

Ferramentas de análise estática baseadas em regras são intensamente empregadas no desenvolvimento de *software* e garantia de código (HOU et al., 2025). Por essa abordagem não necessitar de compilação e execução prévia do sistema, ela é considerada um método rápido e econômico para a identificação de problemas em seus estágios iniciais. Segundo Jiang et al. (2025), as ferramentas emergiram como componentes indispensáveis nos fluxos de trabalho modernos, fornecendo mecanismos automatizados para detecção de defeitos, vulnerabilidades de segurança e violações de padrões de codificação.

Análise estática possui diversas vertentes. Um dos problemas principais dos métodos tradicionais é a diversidade de abordagens necessárias ao lidar com diferentes linguagens. Uma das estruturas mais conhecidas é a Árvore de Sintaxe Abstrata, em inglês *Abstract Syntax Tree* (AST), uma representação intermediária que modela a estrutura hierárquica do código-fonte. ASTs fornecem uma representação de programa geralmente utilizada para análises insensíveis ao fluxo (*flow-insensitive*), onde a ordem sequencial das instruções não é estritamente necessária (MØLLER; SCHWARTZBACH, 2024).

Complementar a isso, utilizam-se os grafos de fluxo de controle, em inglês *Control Flow Graphs* (CFG). Enquanto a AST foca na sintaxe, o CFG é construído para modelar os possíveis caminhos de execução do programa, sendo essencial para análises sensíveis ao fluxo (*flow-sensitive*). CFGs são muitas vezes utilizados para a análise clássica de fluxo de dados, de modo que para cada nó do grafo, se atribui uma restrição que relaciona o valor do nó com seus vizinhos, sejam eles predecessores ou sucessores, para propagar informações ao longo desses nós até atingir um ponto fixo de saída, que a partir destes resultados matemáticos é possível chegar em uma conclusão sobre o programa (MØLLER; SCHWARTZBACH, 2024).

Uma vez que as vulnerabilidades são detectadas, o próximo desafio no ciclo de desenvolvimento é a sua correção. Como uma evolução natural à simples detecção de falhas, o campo de Reparo de Programa Automatizado, em inglês *Automated Program Repair* (APR), tem ganhado destaque na engenharia de *software*. Enquanto a análise estática tradicional se limita a apontar as violações, o APR propõe-se a dar um passo além: gerar automaticamente soluções e aplicar correções de código (*patches*) sem intervenção humana, visando reduzir os custos de depuração (TANG et al., 2021).

Contudo, apesar de sua utilidade, tanto as ferramentas clássicas de detecção quanto os métodos tradicionais de correção enfrentam graves desafios estruturais. Conforme apontado por Hou et al. (2025) e Tang et al. (2021), a eficácia destas abordagens é frequentemente limitada por sua dependência de lógica rígida, o que compromete severamente a sua extensibilidade. A Tabela 1 sintetiza as principais limitações destas abordagens clássicas apontadas na literatura.

Tabela 1 – Limitações da análise estática tradicional

Limitação	Descrição
Escalabilidade e Diversidade de <i>Bugs</i>	A dependência de verificações pré-definidas ou modelagens formais restringe as ferramentas a um subconjunto limitado de padrões de <i>bugs</i> . Isso prejudica a escalabilidade e a capacidade de detectar vulnerabilidades imprevistas ou casos extremos (<i>corner-cases</i>) (YANG et al., 2025a).
Esforço Humano	A criação de regras personalizadas exige um alto nível de especialização técnica. A complexidade sintática torna a tarefa inacessível para desenvolvedores comuns, sendo inviável mesmo com treinamento de curto prazo para realizar uma regra eficiente (HOU et al., 2025).
Falta de Extensibilidade Multilinguagem	A maioria das regras é desenvolvida com lógica rigidamente codificada (<i>hard-coded</i>) para uma linguagem de programação específica. A necessidade de contruir e manter <i>parsers</i> específicos para cada nova linguagem exige alta especialização técnica, o que inviabiliza a portabilidade e gera altos custos na adaptação de analisadores para ecossistemas políglotas (HOU et al., 2025).
Falsos Positivos	Devido à complexidade dos sistemas de <i>software</i> e à riqueza de gramáticas de programação, é inviável que regras manuais prevejam todas as variações estruturais. Essa generalização falha resulta em uma abundância de alarmes imprecisos no mundo real (JIANG et al., 2025).

Fonte: Elaborado pelo autor (2026)

Embora existam outras ferramentas de análise estática que vão além de regras pré-definidas permitindo, através de interfaces amigáveis, que desenvolvedores incluam suas próprias regras personalizadas, como SemGrep, Hou et al. (2025) ressalta como é impossível para um desenvolvedor não especialista desenvolver regras personalizadas eficientes, mesmo com um treinamento de curto prazo. Essa dificuldade é confirmada por pesquisas que revelam que menos de 5% das regras de análise estática utilizadas na indústria são criadas por usuários comuns (HOU et al., 2025).

Além do esforço de criação, as regras de detecção estática são inerentemente imperfeitas (JIANG et al., 2025). Quando é definido o conjunto de regras (*ruleset*), os projetistas focam em semânticas comuns de exemplos conhecidos de *bugs*. Essas observações são abstraídas em modelos generalizados para tentar capturar vulnerabilidades semelhantes em códigos não vistos (JIANG et al., 2025). Porém, Yang et al. (2025a) explica que essa dependência contínua de verificações restritas exige ampla experiência de domínio e um esforço contínuo para manutenção. Como aponta Jiang et al. (2025), a riqueza gramatical da programação torna inviável prever todas as possíveis variações e *corner cases*.

Como resultado direto, estas ferramentas de análise estática tradicionais frequentemente

geram uma abundância de falsos positivos ou falsos negativos no mundo real, além de terem sua escalabilidade severamente prejudicada diante de um aspecto mais amplo de problemas (JIANG et al., 2025; YANG et al., 2025a).

1.1 Contexto

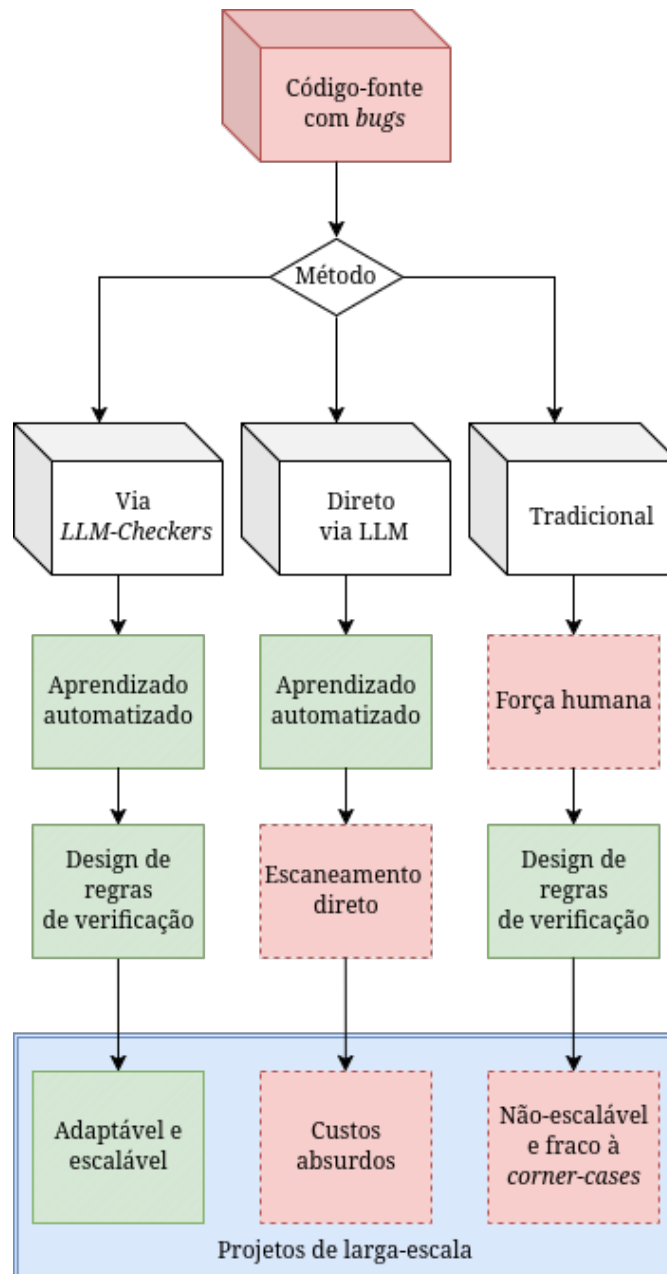


Figura 1 – Motivação para *checkers* feitos por llms.

Fonte: Elaborado pelo autor (2026)

Nos últimos anos, muitas técnicas foram desenvolvidas para minimizar alarmes falsos em analisadores estáticos (JIANG et al., 2025). Recentemente, Modelos de Linguagem de Grande Porte (LLMs) e sua ampla aplicação mudou profundamente o domínio de análise

estática, a maioria dessas técnicas empregando LLMs para compreender o código-fonte diretamente ou os dados das próprias ferramentas de análise estática, para determinar a presença de *bugs*, como sua localização e tipo (HOU et al., 2025).

LLMs podem aprender diretamente de históricos de *commits* de correção (*patch commits*), essenciais para obter o contexto de *bugs* associados ao sistema, a partir que são treinadas em projetos enormes *open-source* com diversas correções de *bugs* (TANG et al., 2021). A capacidade de analisar vários conteúdos sugere que LLMs são capazes de adaptar a novos tipos de falhas sem a necessidade de criação explícita de regras (YANG et al., 2025a).

Contudo, as estratégias de pós-processamento dependem da inferência contínua das LLMs durante a inspeção de código (HOU et al., 2025). Embora as janelas de contexto atuais alcancem marca de milhões de *tokens*, elas ainda impõem limites práticos. O envio (*upload*) de todo código-fonte relevante de uma só vez, somado às repetições necessárias após cada alteração, gera custos computacionais proibitivos e favorece a produção de saídas plausíveis mas incorretas, especialmente com sistemas de grande escala complexos (como exemplo, o *kernel* do Linux, com mais de 50 milhões de linhas de código) (YANG et al., 2025a).

A figura 1 introduz motivações para a necessidade de métodos mais eficientes e, principalmente, especializados, tal que em projetos grandes, a análise estática possui dois desafios: criar regras para diversos padrões de bugs e gerenciamento de bases de códigos imensas (YANG et al., 2025a). O método de análise estática tradicional possui falhas para tratamento de diversos *bugs*, além de trabalho braçal, e análises diretas via LLM possui limites de contexto e custos.

Considerando esses dados, o analisador estático ideal deveria: Detectar uma diversidade de falhas, incluindo corner-cases e semânticas específicas de sistemas, e eficientemente processar milhões de linhas de código (YANG et al., 2025a). Porém, dos métodos existentes, geralmente sacrificam um desses objetivos (YANG et al., 2025a).

O método apresentado por Yang et al. (2025a), KNight, é um analisador estático sintetizado utilizando LLMs, que ao invés de métodos diretos de utilização de LLMs, é realizado um treinamento de padrões de *bugs* a partir de histórico de *patches* que a partir disso é dedicado para um *checker* especializado. A partir desse paradigma, Yang et al. (2025a) sobrepassa os problemas de limites de contextos e custos absurdos de analisar bases de códigos de milhões de linhas com diversos bugs.

1.2 Justificativa

Apesar das conquistas e avanços que Yang et al. (2025a) realizou, Hou et al. (2025) mencionam uma barreira fundamental relacionada à diversidade de linguagens. Até onde sabemos, o KNight é um método direcionado à projetos C/C++ utilizando um *framework*

Clang Static Analysis. A sua expansão para outros ecossistemas exigiria extensos ajustes nesse paradigma (HOU et al., 2025).

A partir dessa limitação, Hou et al. (2025) introduziu a metodologia de *prompting* para abordagem de diferentes linguagens, utilizando poucos exemplos (*few-shot learning*) como aprendizado. Os autores citam a ferramenta *Semgrep*, uma ferramenta utilizada para montagem de regras personalizadas para análise estática, como na Figura 2, tal que a utilizam para o *ruleset* que geram via LLMs.

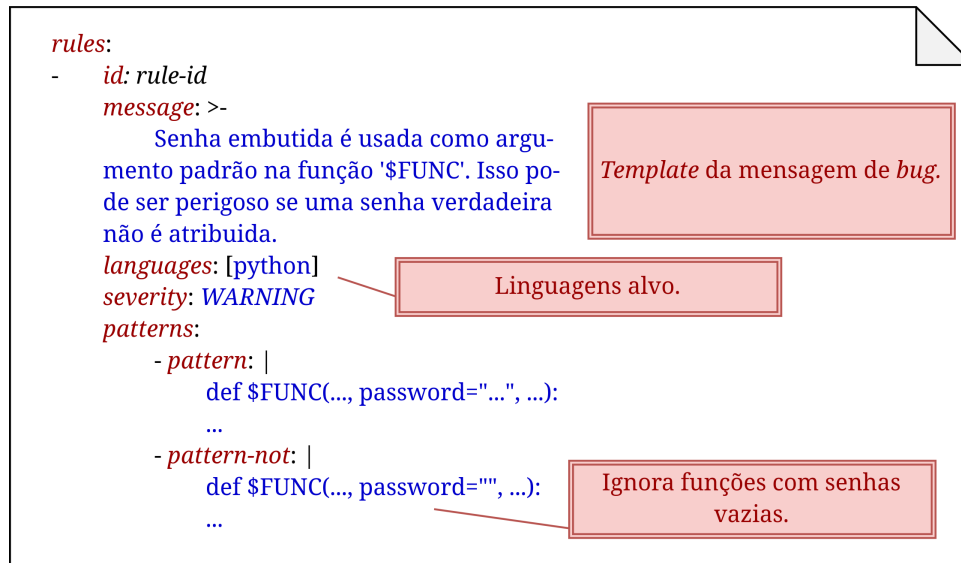


Figura 2 – Exemplo de regra criada para detectar uma senha embutida no argumento da função em Python.

Fonte: Adaptado para português com finalidade de melhorar compreensão (HOU et al., 2025).

A proposta sendo um *framework* baseado em LLMs para regras de análise estática para lidar com as limitações de escrever regras lidando com todos os possíveis casos (*corner-cases*) (HOU et al., 2025). Integrando o domínio de conhecimento adquirido de LLMs durante pré-treinamento não-supervisionado, integrando com as descrições de regras otimizadas para guiar a geração delas pelas LLMs.

Com essa metodologia, Hou et al. (2025) afirma, utilizando a ferramenta *SemGrep* e seu *ruleset* especializado, que seu método atingiu 81,99% de validação gramatical de suas regras pela ferramenta e 74,73% sendo regras funcionais e válidas. Com estes resultados, este cenário apresenta potencial em analisadores estáticos com *frameworks* dinâmicos para sistemas de forma agnósticos à linguagens (*language-agnostic*).

1.3 Hipótese

A hipótese deste trabalho parte de um *framework* possível de inferir principais regras de análise de pelo menos cinco linguagens de programação com pelo menos 80% de acurácia

na ferramenta *Semgrep*, sendo possível realizar uma análise estática com regras definidas específicas para a respectiva linguagem, corrigindo erros gramaticais e especializados.

1.4 Objetivo

Os objetivos desta pesquisa estruturam-se em duas frentes: o objetivo geral, focado em solucionar o problema central do estudo, e os objetivos específicos, que descrevem as etapas práticas e teóricas indispensáveis para a realização desse propósito maior.

1.4.1 Objetivo principal

O objetivo principal do trabalho é desenvolver um *framework* com um agente LLM, que a partir de poucos exemplos (*few-shot learning*) realiza associações com linguagens, utilizando para gerar regras mais utilizadas para projetos associados à linguagem, criando um *ruleset* para *SemGrep* de forma automatizada.

1.4.2 Objetivos secundários

Sobre os objetivos secundários, temos:

1. Implementar um sistema para geração de regras via um agente LLM capaz de identificar a linguagem a partir de pequenos exemplos de códigos, utilizando *few-shot learning*.
2. Validar a eficácia e eficiência das regras para a ferramenta *Semgrep* e seu objetivo de identificar falhas de código, analisando em pelo menos cinco linguagens distintas.
3. Com o treinamento *few-shot*, inferir a gramática para observações mais específicas.
4. Validar as regras geradas no *Semgrep* frente a um conjunto de testes, visando alcançar 80% de precisão na detecção de vulnerabilidades quando comparado às regras nativas da ferramenta.

2 Revisão Bibliográfica

2.1 Inteligência Artificial

A Inteligência Artificial (IA) possui diversas versões segundo pesquisadores, sendo dividido sua definição como a fidelidade de uma performance humana, ou uma definição formal de raciocinalidade. A partir de ambas dimensões, temos quatro possíveis combinações, sendo respectivamente: Agir de forma humana, Pensamento de forma humana, Pensamento racional e Agir racionalmente ([RUSSELL; NORVIG, 2020](#)).

Agir de forma humana, proposto por Alan Turing (1950), tendo o objetivo inicial de descobrir se era possível um computador gerar respostas indiferenciáveis de um humano. Sendo necessário possuir processamento de nossa linguagem, memória do conhecimento, raciocínio e aprendizado se adaptar e identificar padrões ([RUSSELL; NORVIG, 2020](#)).

Pensamento de forma humana, sendo um modelo cognitivo, tal que a partir de uma teoria de como é um raciocínio humano, aplica à um programa computacional. Fazendo parte da área da Ciência Cognitiva, que junta Inteligência Artificial com técnicas experimentais da psicologia para construir teorias precisas e praticáveis sobre a mente humana ([RUSSELL; NORVIG, 2020](#)).

Pensamento racional, utiliza a filosofia de leis de pensamento, onde a lógica exige um conhecimento absoluto sobre o mundo, uma condição que, na realidade, raramente é alcançada. Isso nos permite a construção de um modelo capaz de pensamentos racionais, que a partir de informações cruas para um entendimento de como funciona o mundo ([RUSSELL; NORVIG, 2020](#)).

Agir racionalmente, é definido como um agente racional, agente sendo aquele que realiza uma tarefa, como um programa de computador, porém ao ser racional, permite que ele opere autonomamente, entender seu ambiente e se adaptar à diferentes circunstâncias para realizar objetivos. Um agente racional é aquele que atua para realizar o objetivo da melhor forma, ou quando existe incerteza, a melhor forma possível ([RUSSELL; NORVIG, 2020](#)).

2.1.1 Linguagem Natural

Um sistema formal amplamente utilizado para modelar a estrutura constituinte em linguagens naturais é a Gramática Livre de Contexto (GLC). Também conhecidas como gramáticas de estrutura de frase, as GLCs baseiam-se em um formalismo que é estritamente equivalente à Forma de Backus-Naur (BNF), um padrão universal para a especificação da sintaxe em computação ([JURAFSKY; MARTIN, 2026](#)).

Uma Gramática Livre de Contexto consiste em um conjunto de regras ou produções, tal que cada um expressa a forma em que os símbolos da linguagem podem ser ordenados e agrupados, e um léxico de palavras e símbolos. Uma GLC como essa, define o que chamamos de linguagem formal. Em uma linguagem formal, existe uma divisão estrita: as sentenças (cadeias de palavras) que podem ser derivadas pelas regras da gramática, enquanto as que não podem ser derivadas são agramaticais (JURAFSKY; MARTIN, 2026).

Contudo, essa linha rígida entre o que "pertence" ou "não pertence" à linguagem caracteriza apenas gramáticas formais, servindo como um modelo muito simplificado de como as linguagens naturais realmente funcionam. Ao contrário das linguagens de programação, determinar se uma dada sentença é válida ou faz sentido em uma linguagem natural depende inerentemente do contexto. Na linguística computacional, o uso dessas linguagens formais para tentar modelar a complexidade das linguagens naturais é chamada de gramática generativa, desde que a linguagem passa a ser definida pelo conjunto de possíveis sentenças "geradas" pela sua gramática (JURAFSKY; MARTIN, 2026).

2.1.2 Modelos de Linguagens de Grande Porte (LLM)

Os Modelos de Linguagens de Grande Porte (LLMs), em sua maioria, baseiam-se na arquitetura *Transformer*, originalmente projetada para tarefas sequenciais, como tradução automática. A arquitetura consiste em dois principais componentes: o Codificador (*Encoder*), qual converte uma entrada de *tokens* para uma sequência de vetores representativos (frequentemente chamada de estado oculto ou contexto), e Decodificador (*Decoder*), que utiliza o contexto gerado pelo codificador para gerar iterativamente uma sequência de *tokens* de saída, um *token* por vez (TUNSTALL; WERRA; WOLF, 2022).

Com o passar dos anos, esses blocos funcionais foram adaptados para atuar como modelos independentes. Embora existam centenas de variações de *Transformers*, a maioria se encaixa em um desses três tipos:

Apenas Codificador (*Encoder-only*): Esses modelos convertem uma sequência de texto de entrada em uma representação numérica, sendo muito eficiente para tarefas como classificação de texto ou reconhecimento de entidades nomeadas. A representação computada para cada *token* nessa arquitetura depende tanto do contexto à esquerda (antes do *token*) quanto à direita (depois do *token*), esse mecanismo geralmente denominado como atenção bidirecional (TUNSTALL; WERRA; WOLF, 2022).

Apenas Decodificador (*Decoder-only*): Dada uma entrada de texto (por exemplo "No almoço, eu comi um..."), esses modelos autocompletam a sequência prevendo iterativamente a palavra subsequente mais provável. A família de modelos GPT pertence a essa classe. Nesse caso, a representação computada para um *token* recebido depende exclusivamente do contexto à sua esquerda, esse comportamento é conhecido como atenção causal ou autorregressiva (TUNSTALL; WERRA; WOLF, 2022).

Codificador-Decodificador (*Encoder-decoder*): Utilizados para modelar mapeamentos complexos de uma sequência de texto para outra, esses modelos são projetados para tradução automática e tarefas de sumarização (TUNSTALL; WERRA; WOLF, 2022).

2.1.3 Aprendizado de regras baseado em LLMs

Modelos de Linguagens de Larga Escala (LLMs) as vezes requerem adaptações, como projetos com *APIs* não documentadas ou ferramentas especializadas, essa adaptação, na maioria dos casos, temos opções para estas adaptações como ajustar pesos do modelo (*fine-tuning*) ou utilizar recuperação de exemplos (*few-shot prompting*), sendo abordagens eficazes, porém não possuem suporte para abstrações, reutilização e interpretabilidade de caso a caso (GAO et al., 2026).

No lugar dessas opções, essa abordagem utiliza LLM para induzir e refinar regras lógicas, tais que podem assumir formatos de linguagem natural, lógica de primeira ordem (*FOL*) ou sintaxes específicas, as metodologias a partir dessa abordagem são:

Geração de Regras a partir de Experiência e Falhas, cria-se e ajusta-se regras de acordo com falhas no raciocínio ou exemplos de código para se adaptar ao problema.

- Em Gao et al. (2026), seu treinamento atua coletando "rastros de falha", a partir do código criado e executado, quando resulta em erros ou resultado incorreto, é utilizado como experiência para gerar regras, utilizando o formato "se-então" para esse erro, corrigindo o raciocínio.
- No sistema de Fan et al. (2025) é utilizado a construção de regras baseada em emparelhamento de código traduzido, tal que o aprendizado é realizado baseado em uma estratégia de remoção. Nessa estratégia, a chave é remover um comando específico do programa original e encontrar sua equivalente que desaparece na tradução correta gerada pela LLM.

Consolidação e otimização via MDL (*Minimum Description Length*), a geração de regras via LLM costuma produzir um conjunto extenso ou redundante, sendo necessário uma consolidação para filtragem dessas regras. Em Gao et al. (2026), é utilizado dois métodos: uma heurística via *prompt* para mesclar regras redundantes via LLM, isso consolida objetivamente as regras mas sacrifica complexidade de regras, e o princípio MDL, que busca o equilíbrio matemático entre a complexidade de uma regra e sua eficácia.

- Seu objetivo, nessa configuração, é selecionar um conjunto compacto de regras que corrigem o máximo de erros sem complexidades desnecessárias (GAO et al., 2026).
- O sistema também realiza consolidações gulosas, qual a partir do conjunto total inicial, é podado regras redundantes e generaliza regras específicas até que a biblioteca de regras se torne o mais enxuta e eficaz possível (GAO et al., 2026).

Composição Probabilística Global e Calibração, apesar de LLMs serem excelentes na descoberta semântica de regras, eles falham em agregar múltiplas regras conflitantes ou utilizar pesos probabilísticos durante a inferência baseada puramente em *prompts*. Em [Yang et al. \(2025b\)](#) é feita a combinação da expressividade de LLMs com composições calibradas de modelos probabilísticos.

- Para essa sinergia, o *framework* primeiro utiliza *prompts* para gerar um conjunto de regras candidatas, a LLM é utilizada para analisar o quão específico é o conjunto, além dessa avaliação, as regras são avaliadas por Regressão Logística, um modelo estatístico clássico, calculando o peso probabilístico para cada regra. A partir disso, [Yang et al. \(2025b\)](#) utiliza refinamento iterativo, exercitando o conjunto utilizando "exemplos difíceis" (*hard-examples*), sendo os casos onde obtiveram erros, e os devolve para a avaliação da LLM. Por fim, é comparado este sistema híbrido (LLM e Regressão Logística) com a abordagem tradicional de obter as regras a partir de um único *prompt* para a LLM.

Não finalizado.

2.2 Metodologia da pesquisa bibliográfica

Descreva como a pesquisa bibliográfica foi conduzida, incluindo:

- **Bases de dados consultadas:** Google Scholar, IEEE Xplore, ACM Digital Library, SciELO, Periódicos CAPES, entre outras;
- **Palavras-chave utilizadas:** liste os termos de busca em português e inglês;
- **Critérios de inclusão e exclusão:** quais critérios foram usados para selecionar ou descartar trabalhos (período, idioma, relevância, tipo de publicação);
- **Período de abrangência:** intervalo de anos das publicações consultadas.

Exemplo de descrição:

A pesquisa bibliográfica foi realizada nas bases Google Scholar, IEEE Xplore e ACM Digital Library, utilizando as palavras-chave “machine learning”, “classificação de texto” e “processamento de linguagem natural”. Foram selecionados artigos publicados entre 2018 e 2025, em português e inglês, que abordassem diretamente técnicas de classificação automática de documentos.

2.3 Fundamentos

Aprofunde os conceitos técnicos que serão utilizados na solução proposta. Diferentemente do referencial teórico (que apresenta conceitos gerais), os fundamentos devem detalhar as tecnologias, algoritmos, frameworks ou metodologias específicas que serão empregados no desenvolvimento.

Por exemplo:

- Se o trabalho utiliza aprendizado de máquina, explique os algoritmos escolhidos (árvores de decisão, redes neurais, SVM, etc.);
- Se o trabalho envolve desenvolvimento de software, descreva as tecnologias (linguagens, frameworks, bancos de dados);
- Se envolve análise de dados, explique as métricas e técnicas estatísticas que serão aplicadas.

Utilize figuras, diagramas e tabelas para ilustrar conceitos quando apropriado.

2.4 Trabalhos relacionados

Apresente até **3 trabalhos** que **compartilhem o mesmo foco de resolução de problema** que o seu. O critério principal de seleção é: os trabalhos escolhidos devem ter tentado resolver o **mesmo problema** (ou um problema muito próximo) ao que você está propondo resolver. Não basta que o trabalho aborde o mesmo tema ou utilize a mesma tecnologia — ele deve ter enfrentado o mesmo desafio prático ou teórico.

Por que esse critério é importante? Ao selecionar trabalhos que atacam o mesmo problema, você demonstra que:

- O problema é relevante e já foi reconhecido por outros pesquisadores;
- Existem soluções anteriores, mas com limitações que o seu trabalho pretende superar;
- Seu trabalho não é uma repetição, mas um avanço em relação ao que já foi feito.

Para cada trabalho relacionado, descreva:

1. **Qual problema** o trabalho tentou resolver (e por que é o mesmo problema que o seu);
2. A metodologia ou abordagem utilizada para resolvê-lo;
3. Os principais resultados obtidos;
4. As limitações identificadas na solução proposta;
5. Como o **seu trabalho se diferencia** ou avança em relação a ele.

2.4.1 Trabalho 1: [Título do trabalho]

2.4.2 Trabalho 2: [Título do trabalho]

[Descrever o segundo trabalho seguindo a mesma estrutura acima, sempre evidenciando que o problema abordado é o mesmo ou muito próximo ao seu.]

2.4.3 Trabalho 3: [Título do trabalho]

[Descrever o terceiro trabalho seguindo a mesma estrutura acima, sempre evidenciando que o problema abordado é o mesmo ou muito próximo ao seu.]

Uma tabela comparativa ajuda a sintetizar as diferenças entre os trabalhos:

Tabela 2 – Comparação entre trabalhos relacionados

Critério	Trabalho 1	Trabalho 2	Trabalho 3	Este trabalho
Objetivo	[resumo]	[resumo]	[resumo]	[resumo]
Tecnologia	[tecn.]	[tecn.]	[tecn.]	[tecn.]
Validação	[método]	[método]	[método]	[método]
Limitação	[limit.]	[limit.]	[limit.]	—

Fonte: Elaborado pelo autor (2025)

3 Desenvolvimento

Neste capítulo, apresenta-se a solução proposta para o problema identificado, bem como o cronograma de trabalho previsto para o desenvolvimento completo do projeto no TCC2.

3.1 Solução proposta

Descreva detalhadamente a solução que será desenvolvida para resolver o problema apresentado na introdução. A descrição deve incluir:

- **Visão geral da solução:** o que será desenvolvido (sistema, aplicativo, modelo, ferramenta, etc.);
- **Arquitetura:** como os componentes da solução se organizam e interagem;
- **Tecnologias e ferramentas:** quais linguagens, frameworks, bibliotecas, bancos de dados e ambientes serão utilizados, e por que foram escolhidos;
- **Funcionalidades principais:** o que a solução fará para atender aos objetivos;
- **Metodologia de desenvolvimento:** qual abordagem será utilizada (ágil, cascata, prototipação, etc.);
- **CrITÉrios de validação:** como será medido o sucesso da solução (métricas, testes, avaliação com usuários, etc.).

Utilize diagramas, fluxogramas ou mockups para ilustrar a solução. Exemplo de descrição de arquitetura:

A solução proposta consiste em uma aplicação web composta por três camadas: front-end desenvolvido em React.js, back-end em Node.js com Express, e banco de dados PostgreSQL. A comunicação entre front-end e back-end será feita por meio de uma API REST.

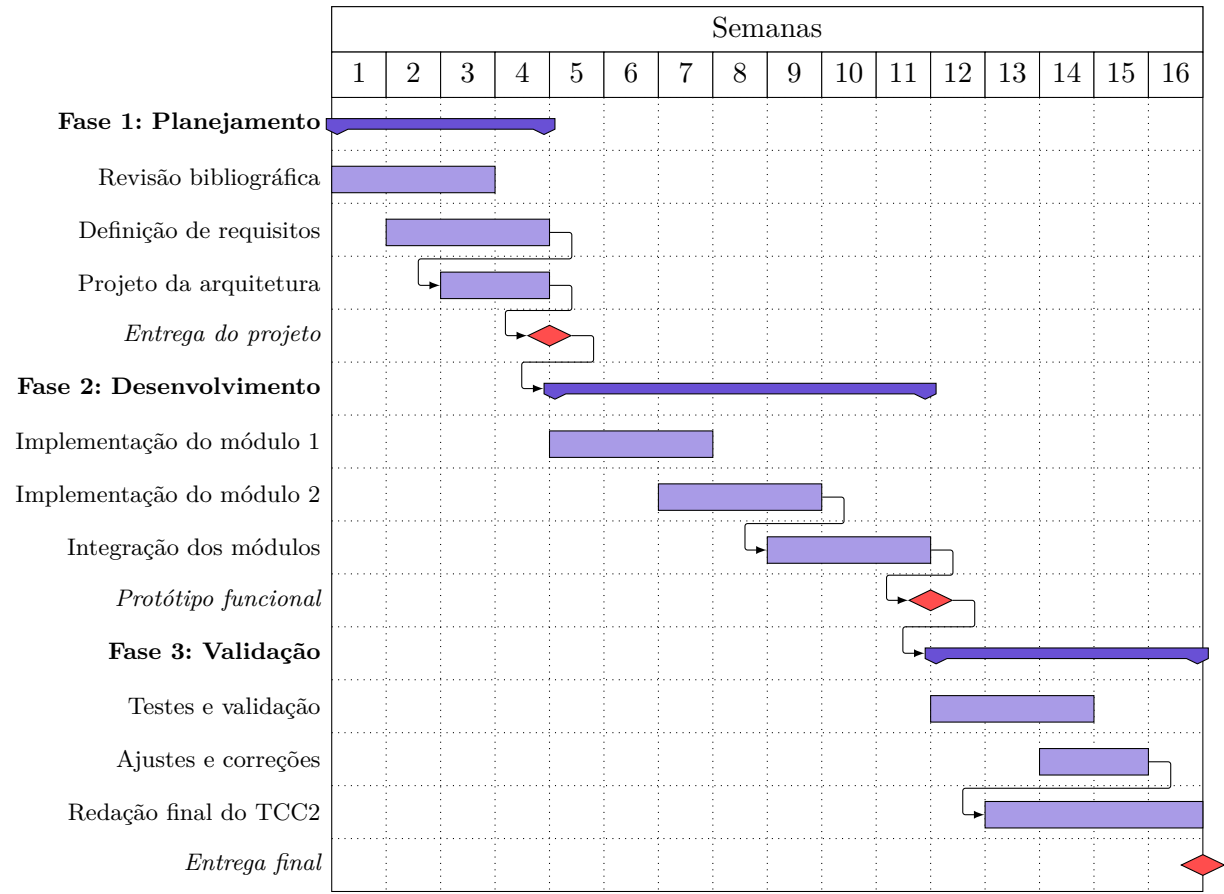
3.2 Cronograma de trabalho

O cronograma a seguir apresenta, no formato de diagrama de Gantt, as atividades previstas para o desenvolvimento do TCC2, distribuídas ao longo das semanas do semestre.

Instruções para personalização do cronograma:

- Ajuste o número de semanas conforme o calendário acadêmico do semestre;
- Modifique as atividades de acordo com as etapas do seu projeto;

Figura 3 – Cronograma de atividades – Diagrama de Gantt



Fonte: Elaborado pelo autor (2025)

- Os marcos (*milestones*) indicam entregas importantes — adapte-os às datas de avaliação;
- As setas entre atividades indicam dependências — uma atividade só inicia após a conclusão da anterior;
- Utilize o diagrama como ferramenta de acompanhamento: atualize-o a cada reunião com o orientador.

Referências Bibliográficas

- FAN, Z. et al. Ruler: Automated rule-based semantic error localization and repair for code translation. In: *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*. [s.n.], 2025. Disponível em: <<https://arxiv.org/abs/2410.02230>>. 12
- GAO, X. et al. Rimrule: Improving tool-using language agents via mdl-guided rule learning. *arXiv preprint arXiv:2601.00086*, 2026. Disponível em: <<https://arxiv.org/abs/2601.00086>>. 12
- HOU, Z. et al. Generation, migration and optimization of cross-language static-analysis rules based on large language models. In: *Proceedings of the 2025 IEEE/ACM International Conference on Software Engineering (ICSE)*. [s.n.], 2025. Disponível em: <<https://ieeexplore.ieee.org/document/11173453/>>. 4, 5, 7, 8
- JIANG, Z. et al. Fact-aligned and template-constrained static analyzer rule enhancement with llms. In: *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. [S.l.: s.n.], 2025. 4, 5, 6
- JURAFSKY, D.; MARTIN, J. H. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. 3rd draft. ed. [s.n.], 2026. Em preparação. Disponível em: <https://web.stanford.edu/~jurafsky/slp3/ed3book_jan26.pdf>. 10, 11
- MØLLER, A.; SCHWARTZBACH, M. I. *Static Program Analysis*. Department of Computer Science, Aarhus University, 2024. Notas de aula. Disponível em: <<https://cs.au.dk/~amoeller/spa/>>. Acesso em: 27 abr. 2026. 4
- RUSSELL, S.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. 4. ed. Hoboken: Pearson, 2020. ISBN 978-0134610993. 10
- TANG, Y. et al. Grammar-based patches generation for automated program repair. In: *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*. Association for Computational Linguistics, 2021. p. 1300–1305. Disponível em: <<https://aclanthology.org/2021.findings-acl.111/>>. 4, 7
- TUNSTALL, L.; WERRA, L. von; WOLF, T. *Natural Language Processing with Transformers: Building Language Applications with Hugging Face*. Sebastopol, CA: O'Reilly Media, Inc., 2022. ISBN 978-1098103248. 11, 12
- YANG, C. et al. Knighter: Transforming static analysis with llm-synthesized checkers. *arXiv preprint arXiv:2503.09002*, 2025. Disponível em: <<https://arxiv.org/abs/2503.09002>>. 5, 6, 7
- YANG, Y. et al. Rlie: Rule generation with logistic regression, iterative refinement, and evaluation for large language models. *arXiv preprint arXiv:2510.19698*, 2025. Disponível em: <<https://arxiv.org/abs/2510.19698>>. 13